

Experiment 9

AIM:

Use semaphores to solve the readers-writers problem with writers being given priority over readers.

Algorithm

Reader Algorithm(Writer Priority)

1. Wait on queue
2. Wait on mutex
3. Increment read_count
4. If read_count == 1, wait on rw_mutex
5. Signal mutex
6. Signal queue
7. Read the shared data
8. Wait on mutex
9. Decrement read_count
10. If read_count == 0, signal rw_mutex
11. Signal mutex

Writer Algorithm(Writer Priority)

1. Wait on queue
2. Wait on rw_mutex
3. Write to the shared data
4. Signal rw_mutex
5. Signal queue

Program

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex;    // Protects read_count
sem_t rw_mutex; // Exclusive access to shared resource
sem_t queue;    // Enforces writer priority

int read_count = 0;
void *reader(void *arg)
{
    int id = *(int *)arg
    sem_wait(&queue);    // Block if writer waiting
    sem_wait(&mutex);    // Locks mutex to safely modify read_count.

    read_count++;
```

```

if (read_count == 1)
    sem_wait(&rw_mutex); // First reader blocks writers

sem_post(&mutex); // Last reader releases writer
sem_post(&queue); // Allow others to queue

// Critical Section (Reading)
printf("Reader %d is READING\n", id);
sleep(1);
printf("Reader %d has FINISHED READING\n", id);

sem_wait(&mutex);
read_count--;
if (read_count == 0)
    sem_post(&rw_mutex); // Last reader releases writer
sem_post(&mutex);

return NULL;
}

```

```

void* writer(void* arg)
{
    int id = *(int*)arg
    sem_wait(&queue); // Writer gets priority

```

```
sem_wait(&rw_mutex);    // Exclusive access
```

```
// Critical Section (Writing)
```

```
printf("\tWriter %dis WRITING\n", id);
```

```
sleep(2);
```

```
printf("\tWriter %dhas FINISHED WRITING\n", id);
```

```
sem_post(&rw_mutex);
```

```
sem_post(&queue);
```

```
return NULL;
```

```
}
```

```
int main()
```

```
{
```

```
pthread_t readers[5], writers[5];
```

```
int id[5];
```

```
sem_init(&mutex, 0, 1);
```

```
sem_init(&rw_mutex, 0, 1);
```

```
sem_init(&queue, 0, 1);
```

```
for (int i = 0; i < 5; i++) {
```

```
    id[i] = i + 1;
```

```
    pthread_create(&readers[i], NULL, reader, &id[i]);
```

```
    pthread_create(&writers[i], NULL, writer, &id[i]);
```

```
}
```

```
for (int i = 0; i < 5; i++) {
```

```
    pthread_join(readers[i], NULL);
```

```
    pthread_join(writers[i], NULL);
```

```
}
```

```
sem_destroy(&mutex);           // Frees semaphore resources.
```

```
sem_destroy(&rw_mutex);
```

```
sem_destroy(&queue);
```

```
return 0;
```

```
}
```